ScaleLockDep: Scalable and Transparent User-space Deadlock Detector

Peizhe Liu
University of Illinois
Urbana-Champaign
peizhel2@illinois.edu

Hieu Nguyen
University of Illinois
Urbana-Champaign
hieunn2@illinois.edu

Lei Huang
University of Illinois
Urbana-Champaign
leih5@illinois.edu

Abstract

ScaleLockDep is a transparent user-space deadlock detector for `pthread`-based programs. Deadlocks are easy to define but hard to reproduce: a dangerous lock order may sit dormant until one particular thread schedule turns it into a hang. A useful detector should catch the execution that is already stuck and also warn about lock orders that make such an execution possible.

ScaleLockDep is implemented as an `LD_PRELOAD` library that interposes on `pthread_mutex` operations. On the blocking path, it keeps ownership and waiting metadata and checks whether a busy mutex would close a live wait cycle. For historical lock-order bugs, it records dependencies among locks and reports cycles in the dependency graph.

A naive backend would update the dependency graph synchronously inside every nested lock acquisition, serializing graph work on the mutex hot path. ScaleLockDep instead offloads graph maintenance to a background worker fed by per-thread ring buffers. On the benchmark suite, it correctly identifies all actual- and potential-deadlock cases with no false positives. On independent-lock workloads at 64 threads, ScaleLockDep sustains 1.6–2.3× overhead, where the same Naive Backend reaches 601×. Our code is published at https://github.com/Winlere/ScaleLockDep

1 Introduction

Locks protect shared state and give programmers a simple critical-section model, but the model becomes fragile once a program has several locks and several paths through the code. One inconsistent acquisition order can be enough to stall every thread that participates in a shared protocol.

Deadlock is also a scheduling problem. The same input may finish cleanly in one run and hang in the next because a different interleaving exposes a circular wait. Lock-order validation [13] addresses this by looking for inconsistent acquisition orders before they materialize as a hang.

ScaleLockDep targets user-space programs built on `pthread` mutexes. It runs as a preloaded shared library, so an application can be checked without source changes, manual annotations, or recompilation. The shim resolves the real pthread functions with `dlsym`, intercepts mutex calls, and updates detector metadata around those calls.

The runtime keeps two kinds of evidence. Current ownership and waiting state support actual-deadlock reports: before a nested lock acquisition blocks, ScaleLockDep walks an alternating sequence of locks and the threads that hold or wait on them, called the *wait chain*, and checks whether the new wait closes a live wait cycle [8]. Historical lock-order evidence supports potential-deadlock reports: acquiring lock L while holding lock H records the order $H \rightarrow L$, and a cycle in that graph marks an unsafe locking discipline. This is the central insight behind Linux *lockdep*, the kernel's runtime locking validator [13].

Maintaining the historical graph is the more expensive of the two. The Naive Backend updates the dependency graph during lock acquisition, so graph traversal sits on the same path as the application mutex operation. ScaleLockDep's ring-buffer backend changes the placement of that work: application threads emit compact lock-order events into per-thread ring buffers, and a worker later deduplicates edges and updates the graph. This report presents the design, implementation, and evaluation of that split.

2 Background

2.1 Deadlock in Lock-Based Programs

Deadlock occurs when a set of execution contexts cannot make progress because each one waits for a resource held by another. In lock-based programs, those resources are usually mutexes. The four Coffman conditions are mutual exclusion, hold-and-wait, no preemption, and circular wait [4]. ScaleLockDep focuses on circular waits caused by inconsistent mutex acquisition order.

A lock-order bug may stay hidden through many test runs. The code has the same bug in every run, but only some schedules arrange the waits into a cycle [12]. This is why the project keeps both current waiting state and historical lock-order state.

2.2 Actual and Potential Deadlocks

We use *actual deadlock* for a live runtime condition. If thread T_0 owns lock A and waits for lock B, while thread T_1 owns lock B and waits for lock A, the current execution is already stuck. The evidence is the ownership and waiting relation at that moment [8].

A *potential deadlock* is weaker but still useful. If one path acquires L_2 while holding L_1, the execution has observed

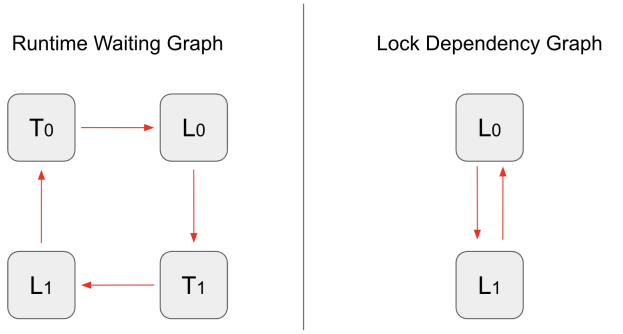

Figure 1. The two graph views ScaleLockDep maintains. Left: the runtime waiting graph contains both threads and locks; a thread-to-lock edge means the thread is waiting on the lock, and a lock-to-thread edge records ownership. The four-edge cycle here is an actual deadlock between T_0 and T_1. Right: the lock dependency graph contains only locks; the two edges $L_0 \rightarrow L_1$ and $L_1 \rightarrow L_0$ summarize lock-order observations across the run, and the cycle marks an unsafe locking discipline regardless of whether any single execution hangs.

$L_1 \rightarrow L_2$. Observing the reverse order later gives a historical cycle. The program may not hang in that run, yet the locking discipline is unsafe because some interleaving can realize the cycle as an actual deadlock [7].

2.3 Graph Views

Figure 1 contrasts the two views. The runtime waiting graph contains both threads and locks. A thread-to-lock edge means that the thread is waiting for the lock; a lock-to-thread edge records ownership. A cycle in this graph corresponds to an actual deadlock.

The lock dependency graph contains only locks. An edge $L_i \rightarrow L_j$ means that some thread acquired L_j while holding L_i. These edges remain after unlock because they summarize past lock-order observations. A cycle in this graph corresponds to a potential deadlock [13].

2.4 Transparent User-Space Interposition

ScaleLockDep uses `LD_PRELOAD` interposition. Calls to `pthread_mutex_lock()`, `pthread_mutex_trylock()`, and `pthread_mutex_unlock()` enter the detector before reaching the real pthread implementation. The application binary stays unchanged [9].

The difference between `pthread_mutex_lock()` and `pthread_mutex_trylock()` matters for live-wait-cycle detection. A blocking lock call can sleep forever once a deadlock has formed. A failed trylock returns immediately, giving the library a chance to inspect the wait chain before it lets the thread enter the blocking call.

3 Design

3.1 Design Goals

ScaleLockDep keeps its target narrow: dynamically linked user-space programs that use ordinary `pthread` mutexes. Within that setting, the library should be transparent to the application, report both live wait cycles and historical lock-order cycles, and keep expensive graph work away from the common mutex path whenever possible.

3.2 Overall Structure

Figure 2 sketches the structure of ScaleLockDep. The runtime sits between the application and the real pthread implementation. A shim handles interposition and forwards successful operations to the detector core. From there, the design splits into two paths. The actual-deadlock path uses current ownership and waiting metadata and runs before a thread enters a blocking mutex call. The potential-deadlock path records lock-order evidence and can run later, after the application thread has moved on.

To use that difference in urgency, application threads summarize nested acquisitions as compact lock-order events and publish them to per-thread ring buffers. A background worker drains those buffers, removes duplicate edges, and updates the dependency graph. The graph still has one owner; the important change is that application threads no longer perform graph traversal while holding up their own lock operation.

3.3 Runtime Metadata

Each observed mutex address is assigned a compact lock slot, and each runtime thread is assigned a thread slot. A lock slot stores the current owning thread slot. A thread slot stores the lock that the thread is waiting for, if any. This is enough to follow a wait chain without materializing a full runtime waiting graph.

Threads also keep local held-lock state. The ordered list is used to update acquire and release state, while the bit-vector form lets the ring-buffer backend build event payloads cheaply. Those local structures describe what the current thread holds; the dependency graph describes what the whole execution has observed over time.

3.4 Actual Deadlock Detection

An actual-deadlock check is only useful before the thread blocks. For a nested mutex acquisition that finds the target mutex busy, the wrapper records the requested lock as the thread's current wait target and follows the wait chain through alternating lock owners and waiting threads [8].

The traversal starts at the requested lock. If that lock is owned, the detector moves to the owner thread; if that owner is waiting, it moves to the next lock. Reaching the original thread means that the new wait would close a live wait cycle.

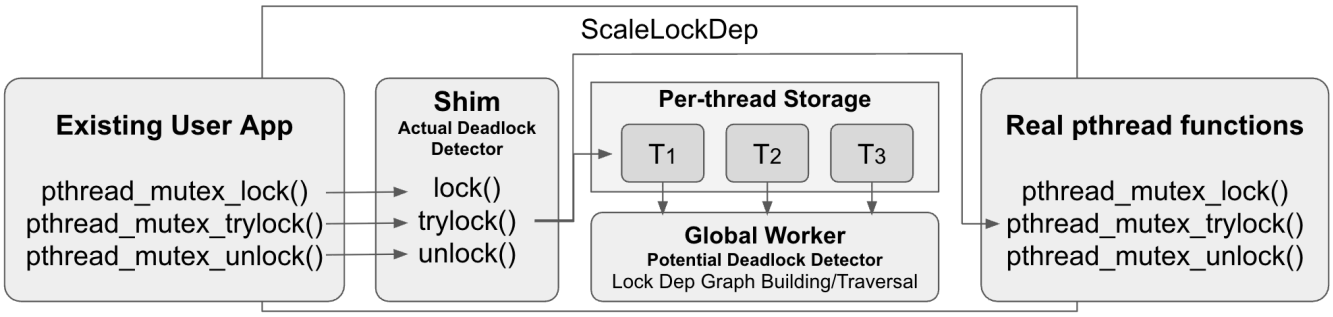

Figure 2. ScaleLockDep architecture. The shim interposes on `pthread_mutex` operations and forwards them to the real pthread functions. Actual-deadlock detection runs synchronously inside the shim before a thread blocks. Potential-deadlock detection is offloaded to an asynchronous pipeline: application threads publish lock-order events into per-thread ring buffers, and the global worker consumes those events to build and traverse the lock dependency graph.

Reaching an unlocked lock or a thread with no wait target ends the search without a report.

This check stays synchronous in both backends. If it were delayed to the worker, it could fire after the application thread has already blocked, defeating its purpose.

3.5 Potential Deadlock Detection

Potential-deadlock detection treats lock order as historical evidence. When a thread acquires lock L while holding locks $H_1, H_2, \ldots, H_k$, the execution has observed the edge batch [13]

$$H_1 \to L,\ H_2 \to L,\ \ldots,\ H_k \to L.$$

The Naive Backend inserts those edges immediately. Before adding $H_i \to L$, it checks whether L can already reach H_i in the dependency graph; if so, the new edge closes a cycle [7]. This gives simple and exact behavior, but every nested acquisition may pay for global graph work.

The ring-buffer backend keeps the same graph semantics and changes the timing. It logs the edge batch as an event and lets the worker update the graph asynchronously.

3.6 Events and Ring Buffers

An event contains a target lock and a bit vector of predecessor locks. For a thread holding $\{H_1, \ldots, H_k\}$ and acquiring L, the event represents

$$\{H_1, \ldots, H_k\} \to L.$$

Unlocks do not create events because releasing a mutex does not erase a historical lock-order dependency.

Each application thread owns one producer side of a ring buffer, and the worker is the only consumer. The common enqueue and dequeue operations use atomic head and tail indices instead of a mutex [11]. This does not make ScaleLockDep lock-free overall: the worker still serializes updates to the dependency graph. It only removes that serialization from the application thread's graph-update path.

3.7 Known Predecessor Filtering

Repeated lock orders dominate many workloads. Once the worker accepts $H \to L$, seeing the same direct edge again adds no information. In the ring-buffer path, the worker publishes a predecessor summary for each target lock so that producers can skip already-known edges.

Before an application thread emits an event, it computes

$$unknown = held\_locks - predecessors[L].$$

An empty set means the acquisition adds no new graph edge. A stale summary may let a duplicate event through, but the worker filters duplicates again before updating the graph. Since a predecessor is added to the summary only after the edge has been accepted, the skip path should not suppress a genuinely new edge.

3.8 Worker-Owned Graph

The worker drains rings, groups events by target lock, merges repeated predecessors, and then updates the dependency graph. For each new edge $H \to L$, it checks whether L already reaches H in the dependency graph by walking the per-lock *reachability summary*, a set recording which other locks each lock can reach. A positive reachability result produces a potential-deadlock report; otherwise the edge is added and the affected summaries are updated.

3.9 Overflow Policy

Bounded rings need an explicit full-buffer policy, and ScaleLockDep offers two. In *stall* mode, a producer waits until the worker frees space, preserving exact potential-deadlock coverage at the cost of backpressure. In *drop* mode, the producer may discard lock-order events under pressure; this lowers overhead but makes potential-deadlock reports approximate.

Actual-deadlock detection does not depend on this policy because it uses the live waiting state and runs before the blocking call.

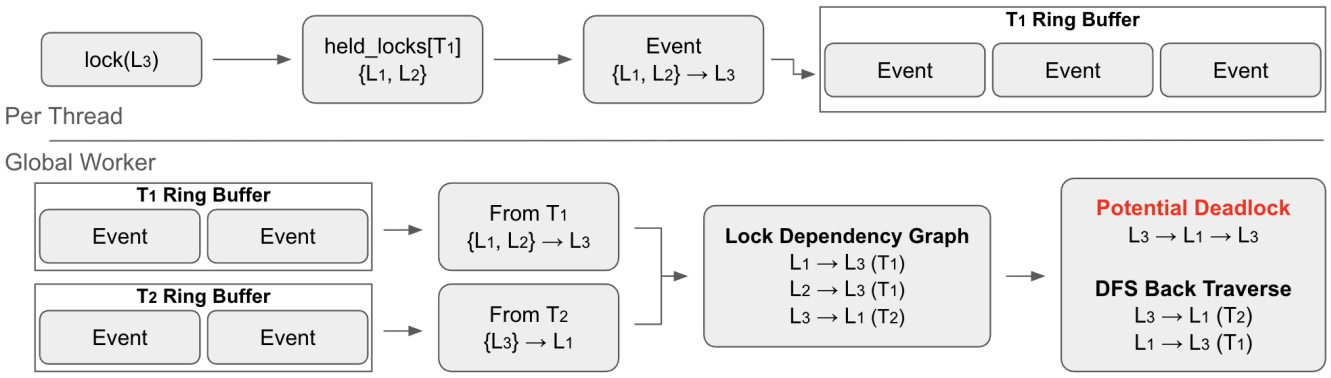

Figure 3. The potential-deadlock event pipeline. When thread T_1 acquires L_3 while holding $\{L_1, L_2\}$, it emits the event $\{L_1, L_2\} \rightarrow L_3$ into its ring buffer. The Global Worker drains the per-thread ring buffers, adds the corresponding edges into the Lock Dependency Graph, and runs a DFS back-traversal from each new edge. Here, a prior event $\{L_3\} \rightarrow L_1$ from T_2 already exists in the graph, so the new edge closes the cycle $L_3 \rightarrow L_1 \rightarrow L_3$ and a Potential Deadlock is reported.

4 Implementation

4.1 Code Organization

The code follows the two-path design. `hook.c` contains the pthread wrappers and resolves the real pthread functions. `core.c` coordinates acquire, release, and before-blocking checks. `state.c` owns lock slots, thread slots, ownership metadata, waiting metadata, and thread-local held-lock state. `graph.c` implements the synchronous dependency graph used by the Naive Backend, while `potential.c` selects the backend and implements the ring-buffer worker. Reports and debug output live in `log.c`.

Keeping these pieces separate made the ring-buffer backend easier to add: the hook layer did not need to know whether potential-deadlock work was handled synchronously or by the worker.

4.2 Initialization and Configuration

At load time, the constructor reads runtime flags, resolves `pthread_mutex_lock()`, `pthread_mutex_unlock()`, and `pthread_mutex_trylock()` with `dlsym`, and initializes the selected potential-deadlock backend. The wrappers use a thread-local recursion guard, so internal library activity can call pthread without reentering the detector. The ring-buffer worker is marked as an internal thread for the same reason.

`LOCKDEP_MODE` selects between the Naive Backend (`global` mode) and the ring-buffer backend (`rb` mode). In `rb` mode, `LOCKDEP_RB_FULL` controls whether a full producer ring stalls or drops potential-deadlock events.

4.3 Wrapper Flow

The lock wrapper has separate top-level and nested paths. If the current thread holds no mutexes tracked by ScaleLockDep, a successful `pthread_mutex_lock()` only needs ownership bookkeeping; a thread with no held lock cannot close a circular wait by blocking. For nested acquisitions, the wrapper first probes the target with `pthread_mutex_trylock()`. An immediate success goes through the ordinary acquire path. A busy result triggers the actual-deadlock check before the wrapper calls the real blocking lock function.

Unlock follows the real pthread result. After a successful `pthread_mutex_unlock()`, the library clears ownership metadata and removes the lock from the thread-local held set. Successful `pthread_mutex_trylock()` calls are recorded as acquisitions; failed trylocks are ignored because they do not block.

4.4 Runtime State

Mutex addresses are mapped to lock slots in a fixed-size table. Runtime threads get thread slots, cached in thread-local storage after the first lookup. A small thread-local cache also speeds up repeated mutex-address lookups on the hot path.

The per-thread held-lock state has two forms. The stack-like list preserves enough information to update acquire and release state, including non-LIFO unlocks. The bit vector supports fast event construction in the ring-buffer backend, where the producer needs the current held set as a predecessor mask.

4.5 Actual Deadlock Implementation

Before a nested blocking acquisition sleeps, `core.c` writes the target lock into the current thread's waiting field and walks the wait chain. The walk uses atomic loads from lock slots and thread slots. Returning to the starting thread produces an actual-deadlock report; reaching an unlocked mutex or a non-waiting thread lets the blocking call proceed [8].

No explicit runtime waiting graph is allocated. The graph is implicit in two fields: owner on each lock slot and waiting target on each thread slot.

4.6 Naive Backend

The `global` mode implements the Naive Backend: it updates the lock dependency graph during acquisition. For every currently held lock H and newly acquired lock L, it tries to add $H \rightarrow L$. A depth-first search checks whether L already reaches H before the new edge is inserted. If that path exists, the new edge completes a potential-deadlock cycle [7].

This backend is useful as a reference point because it keeps all historical graph work synchronous and exact. Its cost is also the bottleneck that the ring-buffer backend tries to remove from application threads.

4.7 Ring-Buffer Backend

The `rb` mode implements the ring-buffer backend, which turns nested acquisitions into edge-batch events. A producer event stores the target lock, the acquiring thread, an optional call site, and a bit vector of predecessor locks that are not already known for that target. The ring has a single producer and a single consumer, so atomic head and tail counters are enough for the common queue operations [11].

The worker drains all registered thread rings, coalesces events by target lock, filters duplicate predecessors, and then updates its dependency graph. New edges update adjacency, reachability, and the published predecessor summaries used by producers. Cycle reports are emitted when the reachability check finds an existing path back to the new edge's source.

When a producer ring is full, `stall` mode waits for space and preserves exact potential-deadlock detection. `drop` mode counts and discards the event, which leaves actual-deadlock detection unchanged but makes historical coverage approximate while the queue is saturated.

4.8 Reports and Logging

Debug logging is controlled by a runtime flag and is disabled during performance runs. Deadlock reports remain enabled. Potential-deadlock reports print the new dependency and the existing chain that made the new edge unsafe. Actual-deadlock reports print the wait chain among thread slots and lock slots.

Optional source-location reporting can be enabled with `LOCKDEP_REPORT_SITES`. When enabled, the library records call sites on the hot path and resolves them on the reporting path. The output is still much simpler than Linux lockdep's reports, but it is enough to explain the cycles used in the benchmark suite [13].

4.9 Implementation Limitations

The prototype supports mutexes only, uses fixed-size tables, and treats mutex addresses as lock identities. The ring-buffer backend also has a single worker-owned graph. These choices keep the implementation compact and make the comparison against the Naive Backend clean, but a production-quality tool would need lock classes, larger dynamic graph structures, and richer reporting [13].

5 Evaluation

5.1 Correctness

To validate the correctness of ScaleLockDep, we built a 30-program test suite spanning three outcome classes: programs that should run cleanly, programs that contain a genuine circular wait, and programs that introduce a graph-level cycle whose actual hang depends on thread scheduling. Table 1 lists each case and Table 2 summarizes the confusion matrix. The suite achieves 100% accuracy, precision, and recall according to our benchmark.

No-deadlock programs. Each of these 13 programs violates at least one Coffman condition [4] by construction. Tests 01–05, 17, 19, and 20 rely on either a single shared resource or consistent global lock ordering, so no back-edge can ever form in the dependency graph. Tests 06, 18, 21, and 22 instead break hold-and-wait: they keep critical sections non-nested or use `pthread_cond_wait`, which atomically releases the mutex before blocking. Test 12 has two threads grabbing the same pair of locks in opposite orders, but each uses non-blocking `trylock` attempts and gives up after a fixed number of retries instead of waiting; the unsafe ordering is structurally present, but since no thread blocks on the other, no real deadlock can actually form.

Deadlock programs. Tests 07, 08, 10, 14–16, 23, 25, 26, 29, and 30 exercise clear ownership cycles, scaling the classic two-thread $A \rightarrow B / B \rightarrow A$ inversion to from 3 to 6-node circular waits and to parallel independent two-thread pairs that share no locks. Tests 09 and 24 model lock-order inversion within a layered hierarchy: test 24 follows the layered-lock inversion pattern surveyed in RacerX [6] (one path acquires the high-layer lock before the low, another inverts the order), while test 09 is the degenerate self-deadlock where a single thread re-acquires its own non-recursive mutex.

Potential-deadlock programs. Four cases build a cycle in the dependency graph whose actual hang depends on scheduling; ScaleLockDep flags the cycle as potential deadlock.

Test 11 (round-robin alternation). Two threads alternate between $A \rightarrow B$ on even iterations and $B \rightarrow A$ on odd iterations under the guise of "balancing" hot-lock contention. Both edges accumulate in the global graph within a single run, so the cycle is flagged on the first conflicting iteration even though no individual thread observes the inversion.

Test 13 (role-based selection). Three threads pick their lock pair by `tid % 3`: T0 takes $A \rightarrow B$, T1 takes $B \rightarrow C$, T2 takes $C \rightarrow A$. The cycle exists only across the union of thread behaviours — no thread carries the inversion locally.

Test	Description	Expected	Detected	Result
test_01	Single thread, one lock	no deadlock	no deadlock	TN
test_02	Two threads, one shared lock	no deadlock	no deadlock	TN
test_03	Two threads, consistent $A \to B$ order	no deadlock	no deadlock	TN
test_04	Eight threads, ordered $A \to B \to C$	no deadlock	no deadlock	TN
test_05	Two threads, disjoint lock sets	no deadlock	no deadlock	TN
test_06	Mostly lock-free, brief sync lock	no deadlock	no deadlock	TN
test_07	Classic 2-thread $A \to B$ / $B \to A$	deadlock	deadlock	TP
test_08	3-thread circular $A \to B \to C \to A$	deadlock	deadlock	TP
test_09	Single thread re-acquires own lock	deadlock	deadlock	TP
test_10	4-thread circular $A \to B \to C \to D \to A$	deadlock	deadlock	TP
test_11	Round-robin rotation looks adaptive	potential deadlock	deadlock	TP
test_12	Trylock retry	no deadlock	no deadlock	TN
test_13	Role-based assignment looks ordered	potential deadlock	deadlock	TP
test_14	2 threads deadlock, 2 run freely	deadlock	deadlock	TP
test_15	Barrier then deadlock in subset	deadlock	deadlock	TP
test_16	One pair deadlocks, third partial	deadlock	deadlock	TP
test_17	Lock convoy, 10 threads on one mutex	no deadlock	no deadlock	TN
test_18	Producer/consumer with condition variable	no deadlock	no deadlock	TN
test_19	16 threads, 4 locks, fixed order	no deadlock	no deadlock	TN
test_20	Per-thread local lock then shared global	no deadlock	no deadlock	TN
test_21	Sequential non-nested acquisition	no deadlock	no deadlock	TN
test_22	Trylock readers with blocking writer	no deadlock	no deadlock	TN
test_23	5-thread pentagon cycle	deadlock	deadlock	TP
test_24	Layered lock-layer inversion	deadlock	deadlock	TP
test_25	6-thread hexagon cycle	deadlock	deadlock	TP
test_26	Two independent 2-thread deadlock pairs	deadlock	deadlock	TP
test_27	Seeded-random per-thread orderings	potential deadlock	deadlock	TP
test_28	Long safe chain with single inversion edge	potential deadlock	deadlock	TP
test_29	Three independent deadlock pairs	deadlock	deadlock	TP
test_30	Staggered deadlock under safe workload	deadlock	deadlock	TP

Table 1. ScaleLockDep test suite results. The horizontal rules separate the suite into thematic test families (single-resource cases, classic cycles, trylock variants, partial-deadlock variants, larger negatives, n-thread cycles, scheduling-dependent cycles, and parallel cycles). Outcome class is given by the **Expected** column; ScaleLockDep reports any cycle as "deadlock" regardless of whether it is currently live or only graph-level.

Test 27 (seeded-random permutations). Four threads each derive a fixed, TID-seeded permutation of four locks via an LCG shuffle and run 30 iterations. Every thread's own ordering is internally consistent, but the threads disagree pairwise, and the first incompatible edge pair trips the detector.

Test 28 (long chain plus single inversion). Five "safe" threads iteratively build the chain $L_0 \to L_1 \to L_2 \to L_3 \to L_4$. One buggy thread acquires L_4 then L_0, contributing the single back-edge that closes a five-edge cycle.

		Predicted	
		Deadlock	**No-Deadlock**
Actual	**Deadlock**	17	0
	No-Deadlock	0	13

Table 2. Confusion matrix over the 30-program suite. Potential-deadlock cases (tests 11, 13, 27, 28) are aggregated into the **Deadlock** row, since ScaleLockDep reports them via the dependency-graph cycle path even though no run hangs.

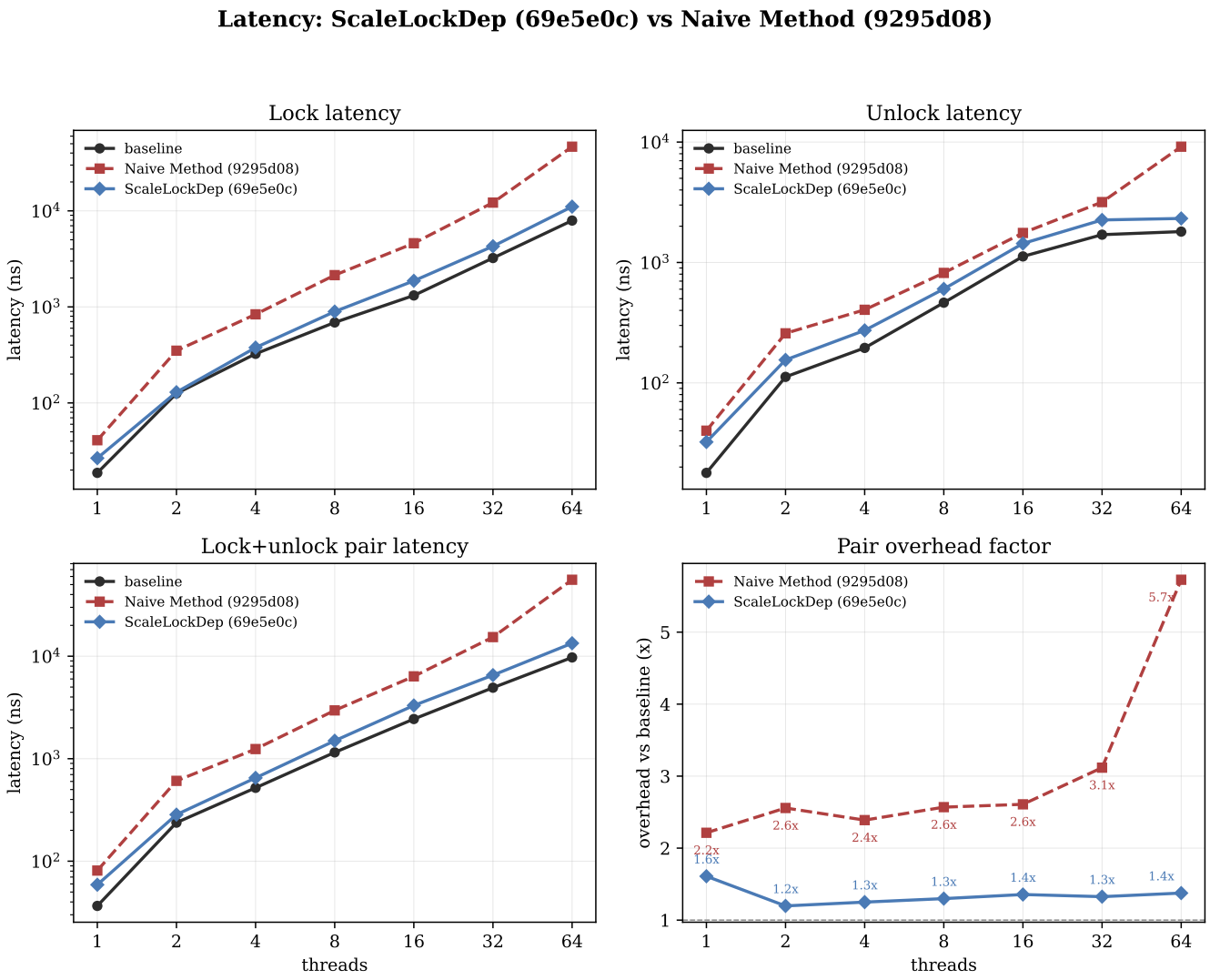

Figure 4. Per-operation latency (ns) vs. threads on a single shared lock with a zero-length critical section. A 15–25 ns measurement cost is included in absolute values.

5.2 Performance

In this section we study the latency overhead (§5.2.1), lock throughput (§5.2.2), nested-lock cost (§5.2.3), and overhead-hiding effect (§5.2.4) of ScaleLockDep. We compare ScaleLockDep against two configurations: an unmodified `pthread_mutex` **baseline**, and a **Naive Backend** that performs equivalent dependency-tracking work but under a single global lock. The Naive Backend isolates the cost of dependency tracking itself from the scalability optimizations contributed by ScaleLockDep.

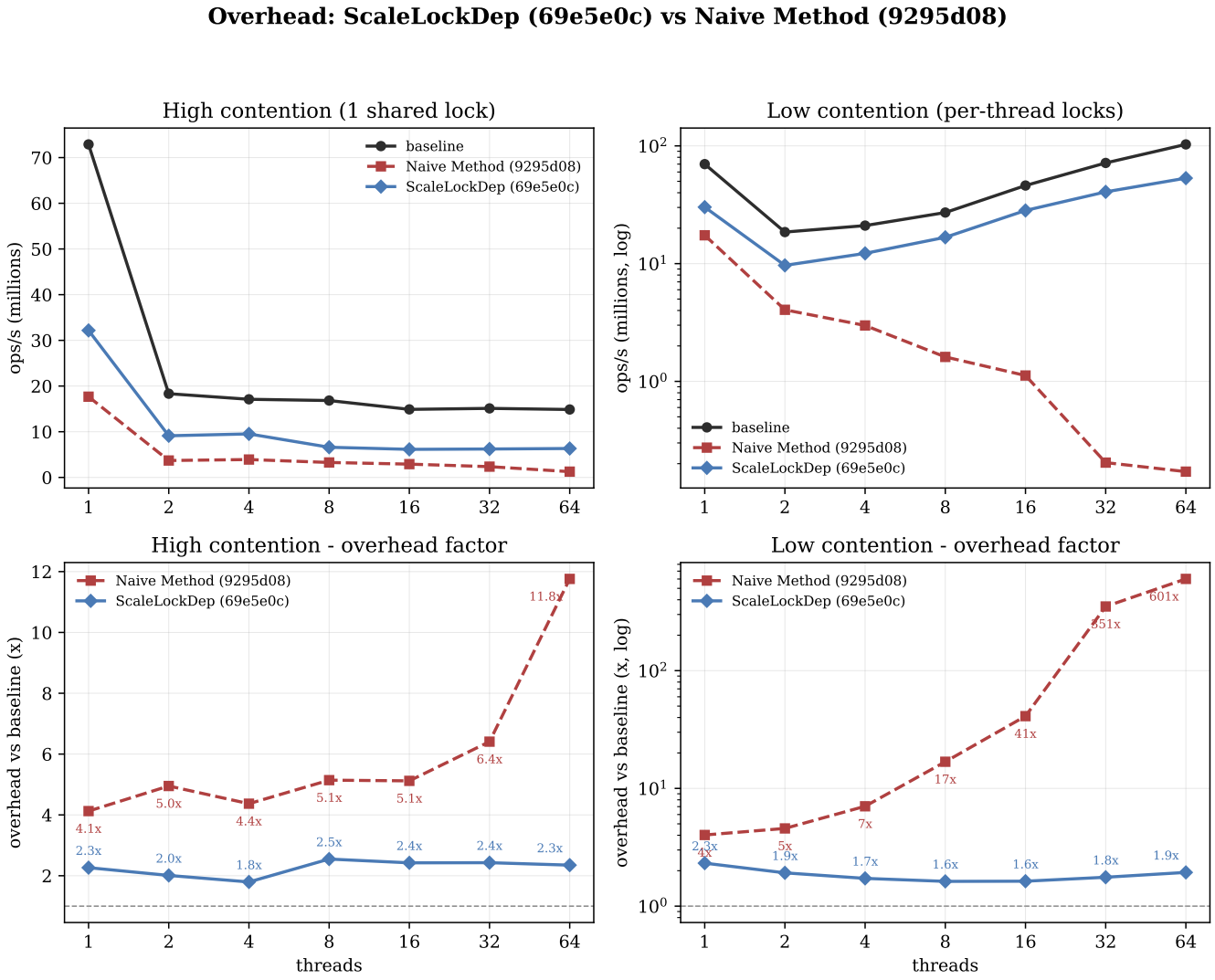

Figure 5. Lock throughput microbenchmarks (million lock/unlock pairs per second) under high contention (single shared lock) and low contention (per-thread locks). Critical section length is zero.

All performance studies were conducted on a machine running `Linux6.19.6-arch1-1x86_64`, equipped with an 11th Gen Intel Core i7-11700 CPU with 8 physical cores and 16 hardware threads, and 16 GiB of RAM. We compiled all benchmarks and the detector with gcc using `-Wall-Wextra-pthread`. ScaleLockDep was injected via `LD_PRELOAD`, intercepting `pthread_mutex` operations entirely in user space. Unless stated otherwise, results use ScaleLockDep at commit `69e5e0c` and the Naive Backend at commit `9295d08`.

5.2.1 Latency Overhead

A lock detector adds latency to every lock and unlock call, directly contributing to thread blocking time and mattering in latency-sensitive workloads. We measure latency in an "n threads competing for 1 lock" scenario where threads repeatedly lock and unlock one shared lock, unlocking immediately after acquisition to suppress the hiding effect studied in §5.2.4. A time-record operation contributes 15–25 ns of measurement overhead per call; absolute values include this cost.

Figure 4 shows that ScaleLockDep adds modest, stable overhead to the lock+unlock pair: between 1.2× and 1.6× across the full 1–64 thread range. The Naive Backend, by contrast, incurs 2.2–2.6× overhead through 16 threads and degrades sharply once threads oversubscribe the 16 hardware contexts, reaching 3.1× at 32 threads and 5.7× at 64. ScaleLockDep avoids this collapse and stays within 1.4× of baseline even at 64 threads, showing that decoupling dependency tracking from a global serialization point is essential under oversubscription.

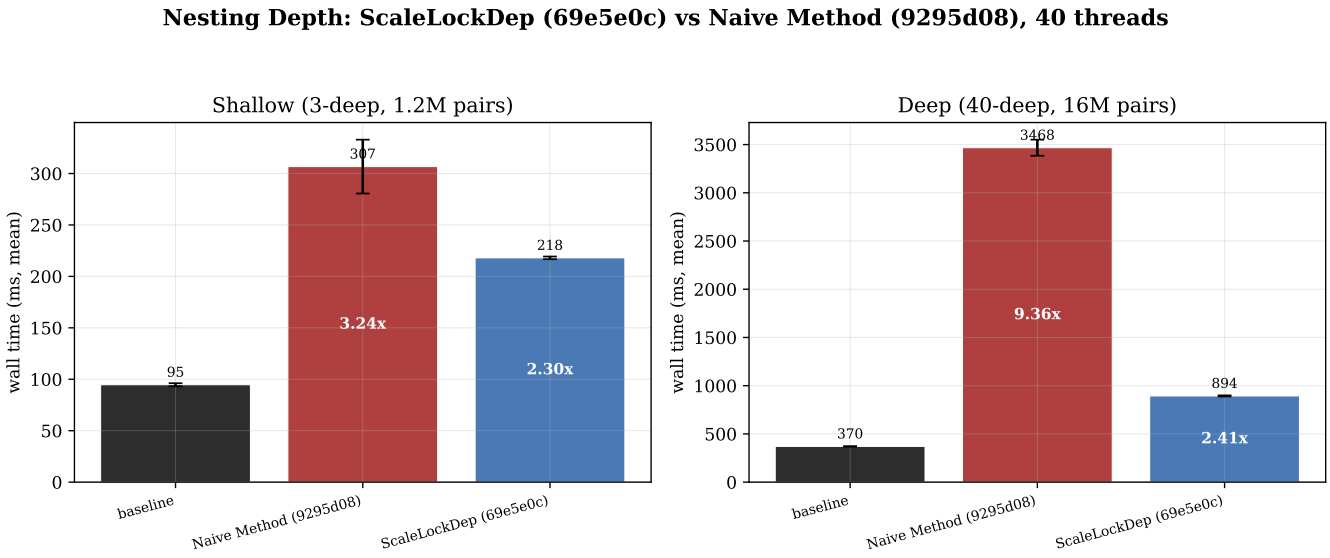

Figure 6. Nested-lock microbenchmark at shallow (3-deep) and deep (40-deep) nesting, 40 threads.

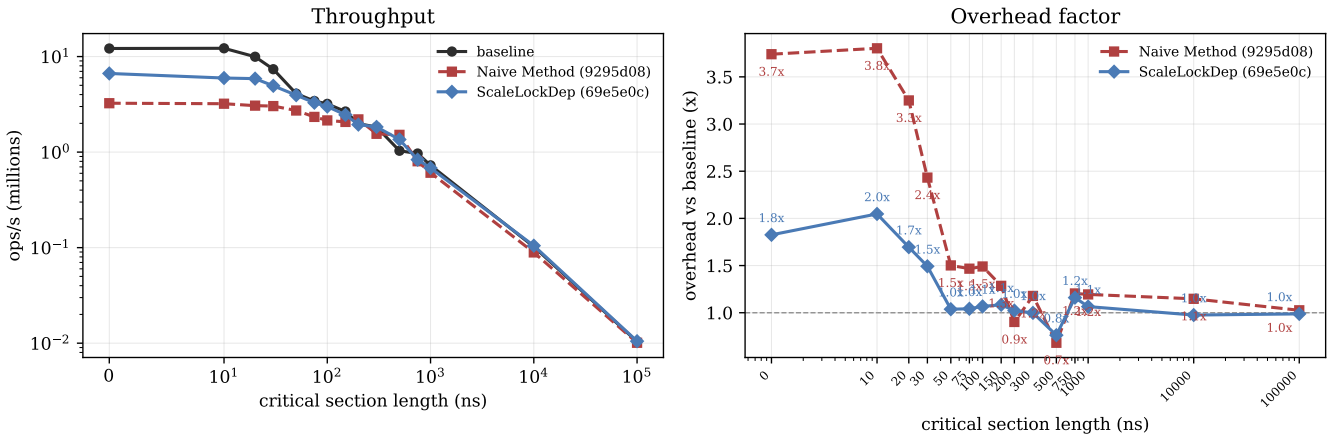

Figure 7. Overhead-hiding microbenchmark: throughput and overhead vs. critical section length at 8 threads on a single shared lock.

Decomposing the pair latency by operation, the lock path bears most of the overhead under both detectors. This is consistent with the dependency check being performed before the lock is granted, whereas unlock only retires the held-set entry.

We also observe a step from 1 to 2 threads visible in all three configurations. This reflects the loss of the cache-coherence "exclusive" state once a second core touches the lock cacheline, and is independent of dependency tracking.

Result Conclusion

ScaleLockDep adds only 1.2–1.6× latency overhead to lock+unlock pairs and remains stable under oversubscription, where the Naive Backend degrades to 5.7×.

5.2.2 Lock Throughput

We next measure the rate of lock operations the detector can sustain. Throughput matters for workloads with very short critical sections (e.g., protected I/O paths) and bounds the rate at which the detector can track activity. We use

two isolated regimes: (a) n threads competing for one shared lock (**High Contention**); (b) n threads each acquiring an independent per-thread lock (**Low Contention**). Threads do no work inside the critical section.

High contention. As shown in the left column of Figure 5, baseline throughput collapses from ~72 Mops/s at 1 thread to ~15–18 Mops/s once a second thread enters, reflecting the irreducible serialization of a single shared lock. ScaleLockDep tracks the same shape at ~6–9 Mops/s, yielding a stable 1.8–2.5× overhead. The Naive Backend ranges from 4.1× at 1 thread to 11.8× at 64, with sharp degradation past the 16-hardware-thread boundary.

Low contention. The low-contention regime is the more discriminating test: the baseline benefits from embarrassingly parallel per-thread locks and scales to ~100 Mops/s at 64 threads. The Naive Backend's global serialization point destroys this parallelism — its throughput *decreases* with thread count, and overhead grows from 4× at 1 thread to 601× at 64. ScaleLockDep instead scales with baseline and holds overhead in a narrow 1.6–2.3× band across the full range. This is the central scalability result: by avoiding a single serialization point, ScaleLockDep preserves the parallelism of independent locks that the Naive Backend destroys.

Result Conclusion

ScaleLockDep sustains 1.8–2.5× overhead under high contention and 1.6–2.3× under low contention, versus 11.8× and 601× for the Naive Backend.

5.2.3 Nested Locks

Dependency tracking work grows with nesting depth, since every acquired lock must be checked against all currently held locks. We evaluate two 40-thread configurations: a **Shallow-Nested** workload of 1.2M lock pairs at depth 3, and a **Deep-Nested** workload of 16M lock pairs at depth 40.

Figure 6 reports wall-clock time. At shallow depth, ScaleLockDep (218 ms, 2.30×) outperforms the Naive Backend (307 ms, 3.24×) by roughly 30%. The gap widens dramatically at deep nesting: the Naive Backend requires 3468 ms (9.36×) while ScaleLockDep completes in 894 ms (2.41×), a roughly 4× improvement. Crucially, ScaleLockDep's overhead is essentially flat across nesting depths (2.30× vs. 2.41×), whereas the Naive Backend's overhead nearly triples as depth grows.

Result Conclusion

ScaleLockDep's overhead is insensitive to nesting depth (2.30× shallow, 2.41× deep), whereas the Naive Backend degrades from 3.24× to 9.36×.

5.2.4 Overhead Hiding in Real Workloads

The microbenchmarks above use a zero-length critical section to isolate detector overhead. In real workloads, lock operations are typically a minor fraction of total work and detector overhead can be hidden by the critical section itself. We measure at what critical-section length the throughput overhead becomes negligible, using 8 threads competing for one shared lock and sweeping critical-section length from 0 to 100 μs.

Figure 7 shows the throughput curves and overhead factor. The curve falls into three regions. Below ~30 ns, the detector dominates and overhead matches the microbenchmark results (1.8–2.0× for ScaleLockDep, 3.3–3.8× for the Naive Backend). Between ~30 ns and the crossover point, the critical section begins to absorb the detector cost. ScaleLockDep reaches near-baseline throughput (1.0–1.1×) at a critical-section length of roughly 50 ns, whereas the Naive Backend requires roughly 200–300 ns to reach the same point. Beyond these crossovers, both detectors track baseline within measurement noise.

In practical terms, ScaleLockDep's overhead is hidden for any critical section longer than ~50 ns — roughly an order-of-magnitude broader operating envelope than the Naive Backend.

Result Conclusion

ScaleLockDep's overhead is hidden once the critical section exceeds ~50 ns, versus ~200–300 ns for the Naive Backend.

5.2.5 Open Source

Our code is published at https://github.com/Winlere/ScaleLockDep. The experiments are conducted on revision 69e5e0c and 9295d08.

6 Related Work

Deadlocks were first systematically surveyed in Coffman et al.'s seminal paper "System Deadlocks" [4], which established the four necessary and sufficient conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait. Lu et al. [12] later examined 105 real-world concurrency bugs in widely-used applications and produced a taxonomy that has informed subsequent detection work.

Research into deadlock detection has been concentrated on two approaches: static or dynamic detection. Towards static detection, Naik et al. [14] presented an effective static deadlock detection algorithm that combines several analyses, each approximating a necessary condition for deadlock, to improve practical precision on Java programs. Breuer and Valls presented a tool that could statically analyze and detect spinlock deadlocks in kernel context [3], which modern tools like DLOS [1] builds on top of with additional improvements like validating code paths to reduce false positives.

Dynamic detection observes the execution itself to flag deadlocks at runtime. Bensalem and Havelund showed that dynamic analysis can catch lock-order cycles that escape

static reasoning by walking the execution trace [2]. UnDead [16] both detects and prevents deadlocks in production by intercepting lock operations to break potential cycles before they form, with sub-3% runtime overhead and no execution-trace storage. Dimmunix [10] grants programs immunity to repeat deadlocks by learning their signatures online and proactively avoiding the same acquisition patterns in subsequent runs.

ScaleLockDep is a dynamic user-space detector that combines lock-order validation for potential deadlocks with runtime wait-chain detection for actual deadlocks. It draws heavy inspiration from *lockdep*, the Linux kernel's runtime locking validator [13], which tracks lock classes, records observed acquisitions, builds a dependency graph, and reports problematic locking patterns including interrupt-context cases. Lockdep's design is tightly tied to the kernel, however; although a user-space port exists [5], it has not become a widely used debugging tool for multi-threaded user programs. ScaleLockDep targets exactly that gap. ThreadSanitizer [15] is a widely used dynamic analyzer that focuses on data races but also reports some deadlocks; it relies on dynamic binary instrumentation via Valgrind, with reported slowdown roughly in the 5–30× range across its operating modes. ScaleLockDep narrows its scope to mutex deadlocks and interposes through `LD_PRELOAD`, trading TSan's broader coverage for transparency to existing binaries and lower overhead.

7 Conclusion

ScaleLockDep demonstrates that a useful deadlock detector can be built entirely in user space for `pthread`-based programs. With `LD_PRELOAD` interposition, the application binary stays unchanged while the library tracks mutex ownership, waiting state, and historical lock order.

The detector's two paths are split by urgency. Live wait-cycle detection has to remain on the blocking path, since a delayed check may arrive after the thread is already asleep. Historical lock-order analysis has more flexibility. Moving that graph work into edge-batch events, per-thread rings, and a worker preserves the same dependency-graph semantics in exact mode while reducing application-side serialization.

The current prototype is still intentionally limited. It supports mutexes only, uses fixed-size tables, identifies locks by address, and relies on a single worker-owned dependency graph. Those limits are acceptable for this course project, but they also point to the next steps: support more synchronization primitives, group lock instances into lock classes, use larger graph representations, and make reports closer to what a developer would need in a real debugging session.

For the workloads in this report, the ring-buffer backend keeps the intended actual- and potential-deadlock behavior while scaling better than the Naive Backend. That is the core result of the project: transparent user-space checking is feasible, but the placement of graph maintenance determines whether the detector remains usable under parallel workloads.

References

[1] Bai, J.-J., Li, T., and Hu, S.-M. DLOS: Effective Static Detection of Deadlocks in OS Kernels. In *Proceedings of the 2022 USENIX Annual Technical Conference (ATC'22)* (Jul. 2022).

[2] Bensalem, S., and Havelund, K. Dynamic Deadlock Analysis of Multi-Threaded Programs. In *Hardware and Software, Verification and Testing: First International Haifa Verification Conference* (2006), vol. 3875 of *Lecture Notes in Computer Science*, Springer, pp. 208–223.

[3] Breuer, P. T., and Valls, M. G. Static Deadlock Detection in the Linux Kernel. In *Reliable Software Technologies – Ada-Europe 2004* (2004), vol. 3063 of *Lecture Notes in Computer Science*, Springer, pp. 52–64.

[4] Coffman, Jr., E. G., Elphick, M. J., and Shoshani, A. System Deadlocks. *ACM Computing Surveys* (Jun. 1971).

[5] Corbet, J. User-Space Lockdep. https://lwn.net/Articles/536363/, Feb. 2013.

[6] Engler, D., and Ashcraft, K. RacerX: Effective, Static Detection of Race Conditions and Deadlocks. In *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles (SOSP'03)* (Oct. 2003).

[7] Havelund, K. Using Runtime Analysis to Guide Model Checking of Java Programs. In *SPIN Model Checking and Software Verification: 7th International SPIN Workshop* (2000), vol. 1885 of *Lecture Notes in Computer Science*, Springer, pp. 245–264.

[8] Holt, R. C. Some Deadlock Properties of Computer Systems. *ACM Computing Surveys* (Sep. 1972).

[9] Jones, M. B. Interposition Agents: Transparently Interposing User Code at the System Interface. In *Proceedings of the Fourteenth ACM Symposium on Operating Systems Principles (SOSP'93)* (Dec. 1993).

[10] Jula, H., Tralamazza, D., Zamfir, C., and Candea, G. Deadlock Immunity: Enabling Systems to Defend Against Deadlocks. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI'08)* (Dec. 2008).

[11] Lamport, L. Concurrent Reading and Writing. *Communications of the ACM* (Nov. 1977).

[12] Lu, S., Park, S., Seo, E., and Zhou, Y. Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics. In *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'08)* (Mar. 2008).

[13] Molnar, I., and van de Ven, A. Runtime Locking Correctness Validator. https://docs.kernel.org/locking/lockdep-design.html, 2006.

[14] Naik, M., Park, C.-S., Sen, K., and Gay, D. Effective Static Deadlock Detection. In *Proceedings of the 31st International Conference on Software Engineering (ICSE'09)* (May 2009).

[15] Serebryany, K., and Iskhodzhanov, T. ThreadSanitizer: Data Race Detection in Practice. In *Proceedings of the Workshop on Binary Instrumentation and Applications (WBIA'09)* (Dec. 2009).

[16] Zhou, J., Silvestro, S., Liu, H., Cai, Y., and Liu, T. UnDead: Detecting and Preventing Deadlocks in Production Software. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE'17)* (Oct. 2017).